

5

Automating Your Development Workflow with Claude Code and GitHub

In this chapter, we will look into the integration between Claude Code and GitHub. This integration involves a series of configuration steps, including installing the GitHub CLI, configuring it to work with Claude Code, and selecting the repositories on which Claude Code will operate.

Once configured, Claude Code can be used directly within a repository. It can be tagged on pull requests and issues, where it reviews code, suggests fixes, and helps resolve reported problems. Claude Code can also participate in pull request workflows by reviewing changes when a pull request is created.

When a pull request is opened, **GitHub Actions** is used to run the automation. GitHub Actions is GitHub's serverless workflow platform, commonly used for CI/CD tasks. In this setup, GitHub Actions executes workflows that rely on the Claude Code CLI and the Claude Code SDK.

For those of you who are not familiar with GitHub Actions, it is an automation system that runs in response to events. These events can include actions such as tagging Claude on an issue or opening a new pull request. Each event can be configured to trigger specific workflows based on project requirements.

The workflows themselves are defined using YAML files, commonly referred to as GitHub workflow files. These files describe which events should trigger an action and which commands or scripts should be executed in response.

This integration is already used in real-world production systems. Anthropic, for example, uses Claude Code with GitHub Actions across its open-source repositories to automate issue handling and related workflows. For a public example, see Anthropic's GitHub Actions workflows:

<https://github.com/anthropics/skills/actions>.

The following topics will be covered in this chapter:

- Installing and configuring the Claude Code GitHub integration
- Tagging Claude Code to resolve GitHub issues
- Rerunning issue resolution with project memory

Installing and configuring the Claude Code GitHub integration

In this section, we will demonstrate the integration using a different repository called IceBreaker (<https://github.com/emarco177/IceBreaker>), which is an open-source project available on GitHub. We are using this repository for demonstration purposes only. You can follow along using any GitHub repository you own. If you choose to use your own repository, make sure you have administrator access, as installing the Claude GitHub app and configuring workflows requires admin permissions.

TIP

Before starting the installation, make sure you are running Claude Code from a directory that already has an initialized Git repository connected to GitHub. In this chapter, we use the IceBreaker repository, which is already configured. The integration will only work inside a valid GitHub repository.

To begin the GitHub integration, open Claude Code from the desktop and type `/git`. This reveals the `/install- github -app` command. Selecting this command starts the setup process.

Before Claude Code can be integrated with a GitHub account, two prerequisites must be met:

- First, the GitHub CLI (also referred to as the `gh` CLI) must be installed locally. At this point, the CLI is not yet installed, so we install it next.
- Second, the GitHub CLI must be authenticated with a GitHub account. Claude Code relies on this authenticated session to interact with repositories.

Installing prerequisites

NOTE

The installation command varies by operating system. The example below shows macOS using Homebrew. If you are using Linux or Windows, refer to the official GitHub CLI installation documentation for the appropriate command for your platform:

<https://cli.github.com/>

To install and authenticate the GitHub CLI, complete the following steps:

1. Open a new terminal instance and install the GitHub CLI using the following command:

```
brew install gh
```

2. Once the installation completes, verify that the CLI is available by running the following:

```
gh --help
```

3. Next, authenticate the CLI with GitHub by running the following:

```
gh auth login
```

During the authentication process, select **HTTPS** as the connection method and proceed with logging in through the web browser. A one-time code is provided, which must be entered in the browser to complete authentication. If two-factor authentication is enabled, approval is completed using the configured second factor.

After this step, the GitHub CLI is authenticated successfully and all prerequisites for Claude Code integration are in place.

Installing the Claude GitHub app

With the prerequisites complete, we return to Claude and continue the installation. We close the terminal window since it is no longer needed and press *Enter* in Claude as instructed. At this point, we are redirected to a GitHub page.

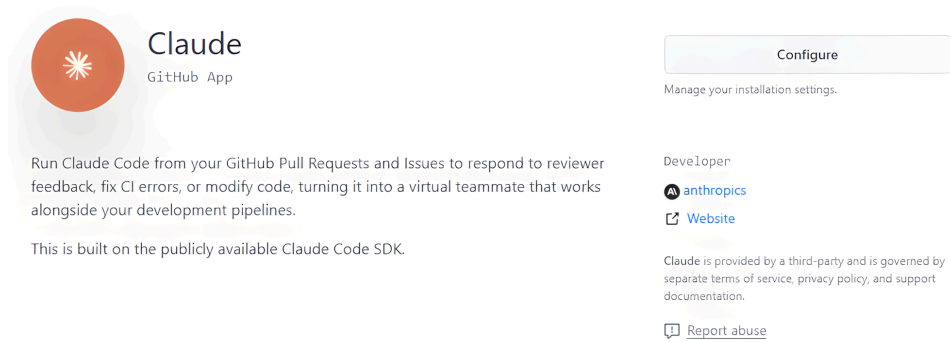


Figure 5.1 – Installing the Claude GitHub app

Start the integration of Claude Code into the GitHub account. This integration allows Claude to comment, create pull requests, and solve issues. We will demonstrate these capabilities later in the book.

After clicking **Install**, we are redirected back to the Claude web application. Returning to the terminal, we press *Enter* again to proceed.

Installing the GitHub workflow

Installing the integration requires adding a GitHub workflow to the repository. This workflow runs the pipelines that handle Claude's comments and actions.

Two options will be presented to you for installation

- **Tag Claude**, which allows us to tag Claude in issues and pull request comments, then runs through a GitHub workflow.
- **Claude Code Review**, which automatically reviews new pull requests

We select the first option.

Next, we choose an authentication method. Since Claude's usage consumes tokens, someone needs to pay for those tokens. We can either configure a long-lived token associated with a Claude subscription or provide an Anthropic API key. The long-lived token is suitable for small to medium usage within a subscription, while an API key is typically used when running large-scale or high-frequency workflows that would exceed subscription limits. In this case, we choose the long-lived token associated with the Claude subscription.

After confirming this choice, we authorize the integration. This authorization also tracks token consumption and associates it with the Claude subscription.

NOTE

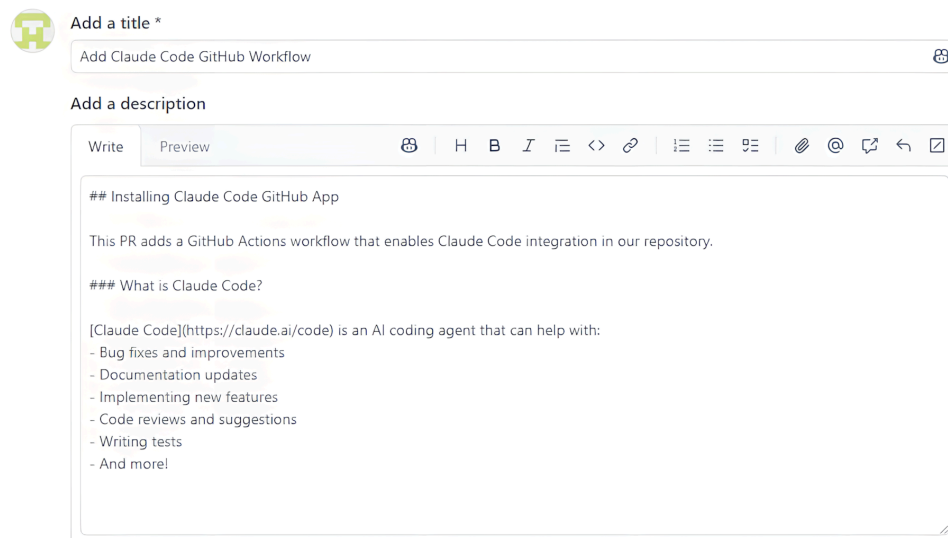
If an error appears, it is because the GitHub integration was initiated from a directory that is not connected to a GitHub repository.

To fix this, open a new terminal in a project directory that already has a GitHub repository initialized and connected to your account. In this example, we use the **IceBreaker** directory, which was introduced earlier in the book and is already configured with GitHub.

Reopen Claude, navigate again to the GitHub installation, and Claude should correctly detect the current repository. Open the configuration popup, select the appropriate account, and grant access to the repositories. After confirming, return to the CLI and configure Claude Code again using the long-lived token.

Creating the pull request

Once authentication completes, Claude initializes the integration by fetching repository information and automatically creating a pull request.



Add a title *

Add Claude Code GitHub Workflow

Add a description

Write Preview

Installing Claude Code GitHub App

This PR adds a GitHub Actions workflow that enables Claude Code integration in our repository.

What is Claude Code?

[Claude Code](https://claude.ai/code) is an AI coding agent that can help with:

- Bug fixes and improvements
- Documentation updates
- Implementing new features
- Code reviews and suggestions
- Writing tests
- And more!

Figure 5.2 – Automatically Creating a Pull Request for Claude GitHub workflow

This pull request contains several updated files under the `.github` directory. These are YAML files that configure the GitHub workflow. We open the pull request to review it.

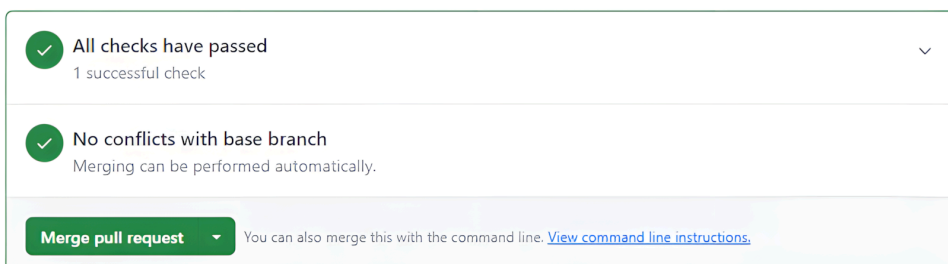
The workflow enables Claude to be tagged in comments and issues and allows it to run through GitHub Actions. GitHub Actions provides the compute environment where Claude operates, and the workflow is configured to use tokens associated with our account.

From a security perspective, the API key or access token is stored as a GitHub Actions secret. This happens automatically during configuration. Only users with the appropriate repository access can trigger the workflow. When Claude is tagged, it can create branches, commits, and comments.

Before merging the pull request, the newly added GitHub workflow should run automatically. In the **GitHub Actions** tab, we will see the Claude Code workflow executing. Once it finishes successfully, proceed with merging.

While the workflow runs, it updates the pull request. Claude reads the files it added, specifically two YAML files under `.github/workflows`: `claude.YAML` and `claude-code-review.YAML`. It reviews these files for security issues and code quality, and posts its feedback directly in the pull request.

At the end of the review, Claude Code approves the pull request, indicating that it finds the changes acceptable, as shown in the following screenshot.



✓ All checks have passed
1 successful check

✓ No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Figure 5.3 – Pull request approved with all checks passed

We now merge the pull request. After merging, the repository should now contain a new directory, `.github/workflows`, which did not exist before. Inside this directory are the two YAML files added by the integration:

- `claude.yml`
- `claude-code-review.yml`

The first YAML file will show the Claude workflow configuration. The file will contain a step called **Run Claude Code**. The authentication token is referenced from a secret named `CLAUDE_CODE_OAUTH_TOKEN`. No sensitive information is stored directly in the workflow file, which keeps the setup secure.

```
name: Claude Code

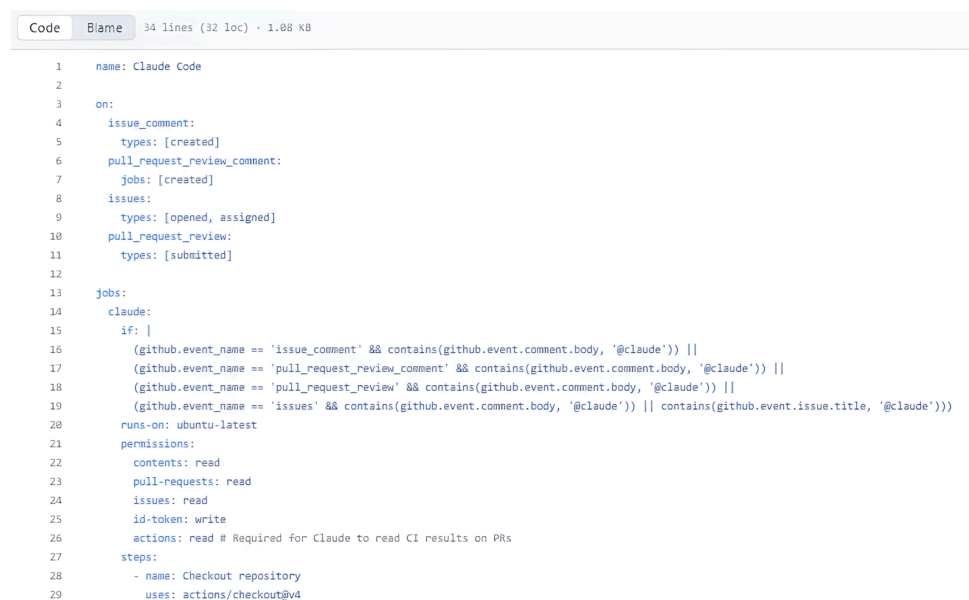
on:
  issue_comment:
    types: [created]

jobs:
  claude:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Run Claude Code
        id: claude
        uses: anthropics/claude-code-action@beta
        with:
          claude_code_oauth_token: ${ secrets.CLAUDE_CODE_OAUTH_TOKEN }
```

A simplified excerpt of the workflow file is shown below.



```
Code Blame 34 lines (32 loc) · 1.08 KB
1 name: Claude Code
2
3 on:
4   issue_comment:
5     types: [created]
6   pull_request_review_comment:
7     types: [created]
8   issues:
9     types: [opened, assigned]
10  pull_request_review:
11    types: [submitted]
12
13 jobs:
14   claude:
15     if: |
16       (github.event_name == 'issue_comment' && contains(github.event.comment.body, '@claude')) ||
17       (github.event_name == 'pull_request_review_comment' && contains(github.event.comment.body, '@claude')) ||
18       (github.event_name == 'pull_request_review' && contains(github.event.comment.body, '@claude')) ||
19       (github.event_name == 'issues' && contains(github.event.comment.body, '@claude')) || contains(github.event.issue.title, '@claude'))
20   runs-on: ubuntu-latest
21   permissions:
22     contents: read
23     pull-requests: read
24     issues: read
25     id-token: write
26     actions: read # Required for Claude to read CI results on PRs
27   steps:
28     - name: Checkout repository
29       uses: actions/checkout@v4
```

Figure 5.4 – Claude GitHub Actions Workflow YAML File

At this point, the integration is complete. The configuration can be further customized, including selecting the model, defining which tools Claude can use, and adjusting other settings to match specific needs.

Using Claude Code to resolve GitHub issues

In this section, we will demonstrate how to tag Claude Code to help resolve issues in a GitHub repository after the integration has been installed.

We start by navigating to the **Issues** section of the repository. The issue we are looking at is a simple one, opened by a user named **happyLo1only**. The issue raised by the user was to improve the readability of a variable name by renaming a variable from `linkedin_username` to `linkedin_url`, since the variable actually holds a LinkedIn URL.

Start by tagging Claude on the issue and asking it to solve the problem:

```
@claude, can you fix this
```

Once Claude is tagged, a new GitHub action starts running automatically. In this case, the action is called **small rename**. This action is triggered as soon as Claude is mentioned.

Open the action to view its details and watch the logs as they stream in real time. The logs consist mainly of JSON output showing tool calls, tool inputs, and tool outputs. While this information is useful for deep debugging, it is not human readable and is difficult to follow at a high level.

For most cases, instead of inspecting the raw logs, it is better to return to the issue itself.

Claude's to-do list and context discovery

Back in the issue view, Claude creates a to-do list that updates as it progresses. The first step Claude takes is attempting to locate a `CLAUDE.md` file in the repository.

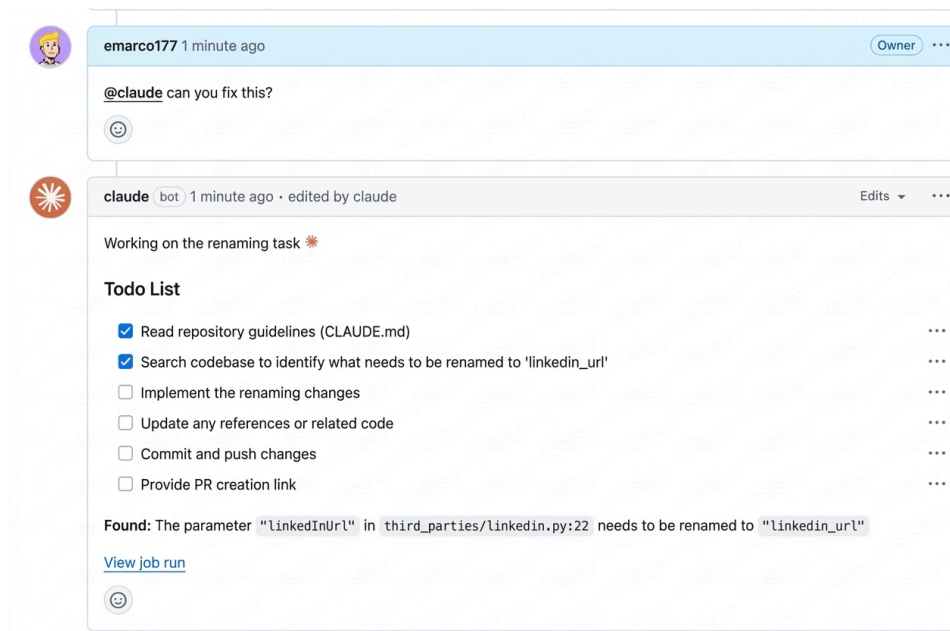


Figure 5.5 – Claude To-Do List and Context Discovery in Issue View

The purpose of a `CLAUDE.md` file is to provide additional context. These files help Claude understand the technology stack, the project structure, coding conventions, and any preferences we want to communicate. This is part of context engineering, where we intentionally supply information to help the model perform its task more effectively.

Spoiler ahead! In this repository, there is no `CLAUDE.md` file. This is intentional to show an important point: in agentic AI workflows, things do not always work correctly, and mistakes are

expected.

Because of this, it is critical to review any changes carefully rather than accepting them blindly. This principle applies throughout all agentic workflows, not just in this example.

Reviewing the proposed changes

Once Claude finishes its to-do list, it updates a file named `linkedin.py`. We then open the **diff** by creating a pull request.

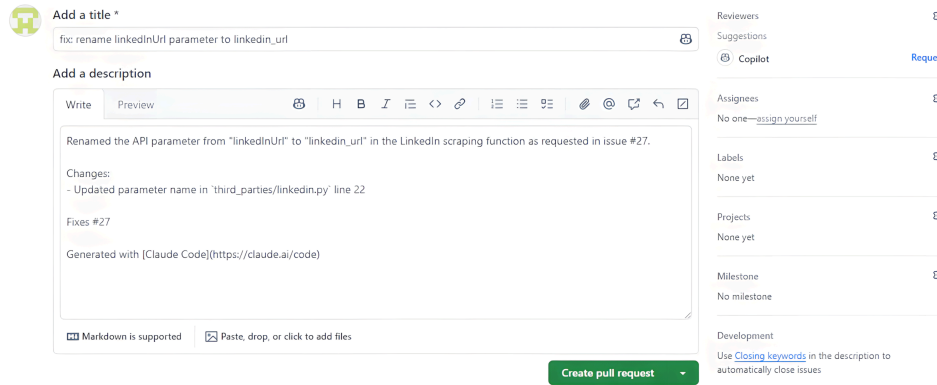


Figure 5.6 – Reviewing the pull request with proposed changes

At first glance, the pull request description looks reasonable. However, when you inspect the actual code changes, you will see that Claude renamed a variable that is part of an API request sent to a third-party service.

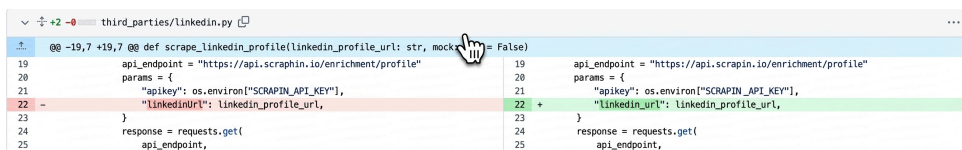


Figure 5.7 – Incorrect Variable Rename Breaking Third-Party API Contract

This is a problem. The third-party API expects the payload to follow a specific format with specific variable names. Changing this variable name breaks the contract with the external endpoint and causes the code to fail.

Because of this, the pull request is incorrect and should not be accepted.

Understanding Claude's workflow

Claude performs its changes on a new branch that it creates automatically. The branch follows a naming convention such as `claude /issue/ < issue-number >`. This is an important detail because it means Claude never works directly on the main branch.

We retain full control over whether to review, modify, or merge the changes. This branching strategy is also foundational for scaling agentic workflows. In practice, we can run multiple Claude instances in parallel, each working on its own branch.

This idea connects to more advanced workflows, such as using Git worktrees, which we will cover later in the book.

This example shows both the power and the limitations of using Claude Code for issue resolution. Claude can automate changes quickly, but without sufficient context, it can introduce

breaking changes. Reviewing code remains essential, and understanding how Claude operates within branches is critical as we scale agentic workflows.

Adding repository context with CLAUDE.md

The goal of this section is to show how adding explicit repository context helps Claude Code produce more accurate and reliable changes when resolving issues.

What if Claude failed to resolve the earlier GitHub issue involving the variable rename because it lacked sufficient repository context? Let us see how to validate that assumption. We start by going to the desktop, where the GitHub project directory is located. We open the project in Cursor and run a quick fetch to pull the latest changes. At this point, we already have the YAML files from the previous step. Next, we open Claude.

Initializing the repository context

We run the `/ init` command in Claude. As mentioned before, this command generates a `CLAUDE.md` file that captures repository-specific context, including the technology stack, project structure, and other relevant details. After confirming the command, Claude begins scanning the repository files. This process takes a short amount of time as Claude gathers the required information.

During initialization, Claude requests permission to execute certain commands, such as listing files. We explicitly allow these actions. A local settings file is updated to whitelist the approved commands, and these permissions persist for the duration of the local session. This approval model provides fine-grained control over what Claude is allowed to do.

Once the scan completes, Claude writes the `CLAUDE.md` file.

Reviewing and committing CLAUDE.md

We locate the newly created `CLAUDE.md` file in the source directory and review its contents. The file includes a structured summary of the repository and appears correct.

We then instruct Claude to commit the `CLAUDE.md` file. Claude prepares a commit message, which we review and accept. The commit is created locally and then pushed to the remote repository. Committing this file ensures that the repository context is versioned, shared with the team, and available to Claude Code when running in automated workflows, such as GitHub Actions.

In the commit history, we should see a new entry indicating that `CLAUDE.md` has been added.

Bash command

```
git commit -n "Add CLAUDE.md for Claude Code guidance"
```

This file provides comprehensive guidance for Claude Code when working with the Ice Breaker codebase, including:

- Development commands for running the app and code quality tools
- Architecture overview and key components
- API dependencies and environment variables
- Important development notes specific to this LangChain application

Generated with [Claude Code](https://claude.ai/code)

Co-Authored-By: Claude <noreply@anthropic.com>
Commit CLAUDE.md with descriptive images

Figure 5.8 – Committing CLAUDE.md to the repository

With the repository context now available, we return to the issue that Claude previously failed to resolve.

We tag Claude on the issue again and ask it to address the problem. This time, we provide an additional hint, specifying that the required change is located within a particular function. We submit the comment:

```
@claude, can you please address this issue? Please notice that you need to edit the function *ice_b
```

A GitHub Actions pipeline starts immediately, confirming that Claude Code has been triggered. We wait for the workflow to complete.

Inspecting Claude's changes

Claude generates a task list and begins execution. Although the experiment is slightly biased due to the additional hint, it still demonstrates the effect of providing proper context.

Once the workflow finishes, inspect the branch created by Claude. The changes show that the variable name has been updated from `linkedin_username` to `linkedin_url`, and the update has been applied consistently both in the function definition and at the call site. Unlike the previous attempt, this change preserves the external API contract and does not break the integration. The logic is correct.

We can now create a pull request with the title **Renamed `linkedin_username` to `linkedin_url`**. This accurately reflects the scope of the change.

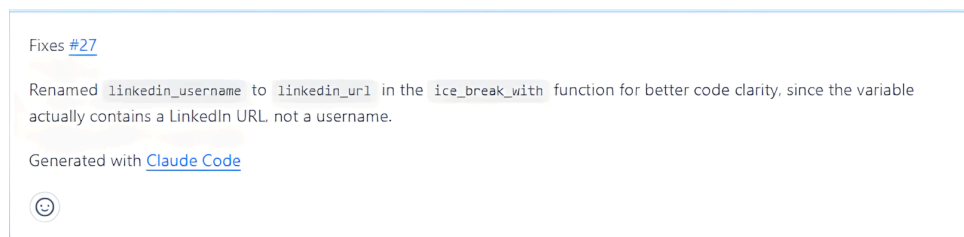


Figure 5.9 – Pull request created with corrected variable name

As soon as the pull request is opened, the Claude Code review workflow runs automatically. Claude analyzes the diff and performs a code review, producing a checklist of review steps and findings.

This review system is fully configurable. Additional checks, such as linters, test execution, or custom validation rules, can be added as needed. This level of configurability is a key reason Claude Code can be effective in team-based engineering workflows.

The review is completed with an overall assessment of approval. No functional issues are detected.

With the review approved, the pull request can be merged.

Summary

In this chapter, we explored how Claude Code integrates with GitHub and how it can be used as part of real development workflows. We walked through installing the required tooling, configuring the Claude GitHub app, and setting up GitHub Actions workflows that allow Claude to respond to issues and pull requests. We also saw how Claude operates by creating branches, opening pull requests, and participating in automated reviews, and why reviewing its changes is essential. Through hands-on examples, we observed how missing repository context can

lead to incorrect changes and how introducing a `CLAUDE.md` file improves accuracy. By now, you have a clear understanding of how to use Claude Code safely, where it adds value, and how to retain full control over your code base.

In the next chapter, we will focus on structured agentic workflows, examining planning mode, spec-driven development, and parallel agent coordination in practice.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.